

Analytical Refactoring: JTMS + Z3 over the Minard Dependency Database

Date: 2026-07-23 **Status:** research direction, parked — investigated so the idea isn't lost; no near-term work planned **Prerequisites:** [purescript-jtms](#) (built 2026-07), Z3 experiments (same week), Minard unified DuckDB schema **Companion docs:** [minard-vision.md](#), [ECOSYSTEM_SYNTHESIS.md](#) (CodeExplorer/docs)

The question

A major part of Minard's original impetus was the belief that we need to see the entanglement of code to judge how hard a refactor will be. Separately, we observed that Haskell/PureScript deliver real *security* of refactoring — when the refactor is over, the code just works — but poor *predictability*: for human and AI alike there is only ever one step of visibility into what breaks when you move things. As the refactor unravels you keep going until you reach a new equilibrium or roll back.

Having now built [purescript-jtms](#) and played with Z3 on the same problem set: **could refactors be done analytically, using these tools over Minard's dependency database?** And, more ambitiously: could the same machinery *propose* rearrangements of a codebase it knows nothing about, semantically, to improve metrics we care about (coupling, build-time dependency)?

Verdict up front: **Part I is a genuinely good research direction with a cheap validation experiment available; Part II is a known research field (software remodularization) where the classic result is disappointment, but PureScript's constraints plus our toolset make a tractable, novel corner of it plausible.** The two parts compose: Part I predicts the *cost* of a proposed move, Part II predicts the *benefit* — together they are the two halves of a refactoring advisor.

Part I — Predicting the refactor wavefront

Diagnosis: the compiler is a one-step TMS with amnesia

The "one step of visibility" phenomenon has a precise explanation. When you change a declaration, the compiler performs exactly one wave of invalidation and *reports only the frontier* — and error masking (errors hiding errors, module compile order hiding whole modules) means even that frontier is partial. The compiler never reports:

1. the **full transitive extent** of invalidation, and
2. which invalidated sites are **re-justifiable locally** (the call site absorbs the change) versus which **propagate** (the site's own interface must change, pushing the wave outward).

The second distinction is the whole game: the wavefront's behaviour at each site determines whether the refactor converges or unravels, and today we learn it one compile cycle at a time.

The JTMS reframing

A justification-based truth maintenance system is precisely the machine for this.

- **Claim per declaration:** "*D compiles at interface T.*"
- **Justification:** the interface facts of the declarations D depends on, plus the *shape* of D's use of each.
- **Retract** the premise you intend to change; **propagate**; read off the entire out-set before touching a file.

The `purescript-jtms` API already has the right primitives:

JTMS primitive	Refactoring meaning
<code>alsoWhy</code> (alternative justifications)	"this site can be re-justified without changing its own interface" — the converge/propagate distinction
<code>explain</code> / <code>axiomsBehind</code>	provenance: <i>why</i> this distant thing broke (the compiler gives errors; this gives the chain)
<code>saturate</code> / <code>learn</code>	live re-propagation as the refactor executes and new interface facts are asserted

The `jtms-make` demo (hylograph-demos) is already this shape — build-graph invalidation with proof DAGs — pointed at Makefiles instead of modules. An in-house template exists.

The crux: precision lives in edge labels, not reachability

Plain transitive closure over depends-on is worthless — that is the hairball Minard already draws, and it wildly over-approximates (changing a *body* breaks nothing downstream; changing a *type* breaks only incompatible uses). All the analytical value is in a classification:

(kind of change) × (shape of use) → survives / breaks-but-locally-fixable / propagates

Change taxonomy (each row has a different propagation rule per use-shape):

- rename; delete
- arity change; argument type change; return type change
- constraint added / removed
- constructor added (breaks only exhaustive matches); constructor removed
- record/row field added / removed / retyped
- instance added / removed (see failure modes)

This is Chianti-style change impact analysis (Ren et al., ~2004, for Java) — but nobody has done it for a pure language with explicit signatures, where it is far more tractable: **top-level type signatures firewall inference at declaration boundaries**, so the wave mostly cannot sneak around the graph. The analysis is most precise exactly where the code is well annotated — an incentive gradient, not a limitation.

Minard schema audit (2026-07-23)

Checked `minard/database/schema/unified-schema.sql`:

Already present (the type side is surprisingly well stocked):

- `declarations.type_ast` — full structured type AST per declaration (from docs.json)

- **child_declarations** — constructors with **constructor_args**, instances with **instance_constraints**, class members
- **function_calls** — caller→callee edges with **source_span** and **call_count** (from corefn.json)
- **commits** / **module_commits** / **declaration_metrics** — churn, for weighting and for the validation experiment

The gap (the one concrete build item):

- **function_calls** records *that* A calls B, not *how*: applied arity at the site, value vs type position, whether constructors are pattern-matched (exhaustively?), whether the use is mediated by a class method. **One CoreFn extraction pass to add use-shape labels to call edges is the whole missing substrate.**

Where Z3 fits — a different role from the JTMS; we want both

- **JTMS = propagation + explanation + incrementality.** "Retract this premise: here is everything that goes out, with justification chains." A live companion *during* the refactor: as new interfaces are asserted, it re-propagates.
- **Z3 = finding the new equilibrium before starting.** Encode:
 - *hard constraints*: every surviving justification must be satisfiable;
 - *soft constraints*: "don't touch declaration D", weighted by cost (declaration_metrics / churn supply weights).

MaxSAT yields the **minimal repair set** — "this refactor's true price is these 23 declarations". **Unsat cores** yield the other answer we currently learn only after days: "no equilibrium exists unless X also changes" — the roll-it-back verdict, computed up front. Varying soft-constraint weights yields *alternative* equilibria, so two refactor plans can be compared analytically.

The compiler remains the oracle; this is a **prediction layer**, so it need not be sound — it needs to be useful. An 80%-accurate blast radius with a confidence gradient changes the decision calculus at the moment of the first edit.

What this does for Minard

It upgrades "see the entanglement" from a static picture to a picture with *semantics*: colour every node by predicted fate (out / locally re-justifiable / safe / unknown), render the propagation wavefront, click a doomed node and get its **explain** DAG. The entanglement view finally answers the question it was built to ask.

Honest failure modes

- **Type class instance resolution** — the hard one. Adding an instance changes resolution non-locally, and "who relies on this instance" is invisible in a call graph. Start by marking instance-mediated edges *unknown* rather than pretending.
- **Row types / records** — "who touches this field" is structural, not nominal; **type_ast** helps but the call graph alone won't catch it.
- **Compounding uncertainty** — each edge classification carries error; a five-hop prediction multiplies it. Hence confidence gradients, never oracle claims.

- **Unannotated code** — inference ripples the graph doesn't capture. Mitigated by house style (signatures everywhere).

The experiment that settles it cheaply

Ground truth already exists: git history contains real refactors that unravelled, and Minard has the commits layer.

1. Pick 3–5 historical refactors (multi-commit episodes that began with an interface change).
2. Take the *first* commit's edit as the retracted premise.
3. Run the propagation — even a hand-rolled or recursive-SQL version with crude edge rules, **before** building the use-shape extractor.
4. Score predicted touched-set against the actual final diff (precision/recall).

This tells us whether the classification rules can be tight enough to matter before we invest in the extractor, the Z3 encoding, or any UI.

Part II — Semantics-free rearrangement proposals (the ambitious ask)

Question: knowing nothing of a codebase's semantics, could these tools analytically propose rearrangements that reduce coupling, or improve some other desirable metric (maintainability, build-time dependency)?

This is a known field, with a known trap

"Software remodularization" / "architecture recovery" has ~25 years of literature: Bunch (Mancoridis et al.) hill-climbs a Module Dependency Graph against MQ (cohesion/coupling quality); community detection (Louvain/Leiden modularity) gets used the same way; MoJo distance scores recovered decompositions against expert ones. The classic negative result: **metric-optimal partitions are routinely rejected by the people who know the code** — they maximize the objective while being conceptually incoherent, and different runs produce unstable partitions. Pure graph metrics do not capture *aboutness*.

Two semantics-free signals partially rescue it:

- **Evolutionary (logical) coupling** — files/declarations that change *together* in git history belong together (Gall et al.; popularized by Tornhill). This is revealed human semantics, extracted without understanding the code. Minard's commits layer already holds the data.
- **Naming vocabulary** — identifier/module-name similarity as weak topic signal.

Why PureScript makes the problem unusually tractable

The type system removes most of the degrees of freedom that make remodularization ill-posed elsewhere. Legality of a rearrangement is *decidable from the database*:

1. **Module graph must be a DAG** (compiler-enforced, no cross-module recursion) → any mutually recursive declaration group (SCC in the call graph) **must be co-located**. Hard constraint, computable today from `function_calls`.

2. **Orphan-instance rule** — an instance must live with its class or its type head → instance placement is a forced 2-choice. Constructors travel with their data declaration; class members with their class.
3. **Re-exports** preserve the public API while internals move → rearrangements can be externally non-breaking by construction.
4. The freely movable atoms are value declarations; everything else is constrained.

So the search space is *partitions of declarations into modules, subject to hard legality constraints* — a constrained graph-partitioning problem, which is exactly MaxSAT/ILP territory.

Objectives that are well-posed without semantics

- **Cut minimization / MQ**: fewer cross-module edges, higher intra-module density.
- **Expected rebuild cost** — the sleeper, and directly valuable: $\sum_d \text{churn}(d) \times |\text{downstream module cone of } d\text{'s module}|$. PureScript's rebuild unit is the module; a change rebuilds the downstream cone. Moving *hot* declarations out of widely-imported modules (the "utils module with one hot function" pathology) measurably cuts build time. Minard has both churn and the cone. This objective is honest: it optimizes a quantity we actually pay.
- **Interface narrowing**: distinct names imported per module edge.
- **Reach / transitive metrics** Minard already computes.

The realistic shape: move-recommendation, not global re-partition

Global optimal partitioning at ecosystem scale (thousands of declarations) is out of Z3's comfortable range and — per the literature — not even desirable. The tractable, useful version:

- **Greedy/local**: for each declaration, enumerate *legal* destination modules (hard constraints above), compute the objective delta, rank ecosystem-wide top-k single moves. Incremental delta evaluation is TMS-shaped (justifications for "edge e is cross-module", "module A imports B").
- **Z3 on bounded subproblems**: splitting *one* god-module into k coherent submodules is a few-hundred-node partition with an acyclicity side condition — within MaxSAT/ILP range. Use the solver where the problem is small and hard, not large and soft.
- **Human in the loop for coherence**: the machine proves legality and computes metric deltas *with provenance* ("this move cuts 14 cross-module edges and shrinks the rebuild cone of hot declaration X from 89 modules to 12"); the human judges aboutness. Weight proposals by co-change and naming similarity to pre-filter the conceptually absurd ones.
- **Direct manipulation**: in Minard, drag a declaration onto another module and see predicted metric deltas live — the Hylograph control-surface ethos applied to the codebase itself.

The synthesis: the two parts are one advisor

Part II proposes a move and computes its **benefit** (metric deltas, legality). Part I computes its **cost** (predicted wavefront: which call sites break, which absorb, minimal repair set). A proposal is only actionable when benefit is computed *and* cost is bounded. Same database, same TMS machinery, two directions of the same analysis — and both render in Minard.

Honest assessment of Part II

Not impossible; *partially* solved elsewhere and known to disappoint when done as pure metric optimization. The novel, plausible corner: **declaration-granularity move recommendation for a pure language, where legality is decidable from the type structure, the objective includes measured build cost, proposals carry provenance, and a human (or the Part I engine) prices each move before it's taken.** Nobody has built that, and most of its substrate already exists in this ecosystem.

If/when this gets picked up

1. **Zero-build validation** (days): replay 3–5 historical refactors with recursive-SQL propagation and crude edge rules; measure precision/recall.
2. **Use-shape extraction** (the one build item): CoreFn pass labelling `function_calls` edges with applied arity, value/type position, pattern-match exhaustiveness, class-method mediation.
3. **JTMS encoding**: declarations as claims, use-shapes as justifications; wavefront + `explain` in Minard.
4. **Z3 planning layer**: MaxSAT minimal repair sets; unsat-core "no equilibrium" verdicts.
5. **Part II pilot**: rebuild-cost objective + legal-move enumeration on one repo; top-k move proposals with provenance; judge by eye.

Related work (pointers, not a survey)

- Ren, Shah, Tip, Ryder, Chesley — *Chianti: change impact analysis for Java* (OOPSLA 2004): the atomic-change-classification precedent.
- Doyle 1979 (JTMS); de Kleer 1986 (ATMS — multiple-context "what if I also change Y" exploration is the natural extension).
- Mancoridis et al. — *Bunch* (remodularization by metric search); MoJo distance.
- Gall, Hajek, Jazayeri — evolutionary coupling from release history; Tornhill, *Your Code as a Crime Scene* (churn × coupling hotspots).
- Provenance semirings (Green et al.) — the justification structure *is* why-provenance of "X compiles"; relevant if the propagation is ever done as recursive SQL in DuckDB rather than in the JTMS.